

OPERATING SYSTEM CONCEPTS LAB EXPERIMENTS

EXP - 08: BANKERS DEADLOCK AVOIDANCE ALGORITHM

```

user123@localhost:~/bankers
[user123@localhost ~]$ ls
abc.sh  awk  Desktop  Downloads  ifelse.sh  memory  page  Pictures  Public  REC  sample.sh  scheduling  SFA  system_calls  Videos
ab.sh  bankers  Documents  Folder  loops.sh  Music  pc  posneg.sh  qns.txt  sample.c  sample.txt  script  sh  Templates  while.sh
[user123@localhost bankers]$ cd bankers
[user123@localhost bankers]$ vi bankers.c
[user123@localhost bankers]$

```

```

user123@localhost:~/bankers — /usr/bin/vim bankers.c
#include <stdio.h>

int main(){
    int n, r, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &r);

    int alloc[n][r], max[n][r], need[n][r], avail[r];
    int finish[n], safe_sequence[n];
    int count=0, safeFound;

    //Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for(int i=0; i<n; i++){
        for(int j=0; j<r; j++){
            scanf("%d",&alloc[i][j]);
        }
    }

    //Max Matrix
    printf("Enter Max Matrix:\n");
    for(i=0; i<n; i++){
        for(j=0; j<r; j++){
            scanf("%d",&max[i][j]);
        }
    }

    //Available Matrix
    printf("Enter Available Resources:\n");
    for(i=0; i<r; i++){
        scanf("%d", &avail[i]);
    }

    //Initialize finish array
    for(int i=0; i<n; i++){
        finish[i]=0;
    }

    //Calculate Need Matrix
    for(i=0; i<n; i++){
        for(j=0; j<r; j++){
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    //Bankers Algorithm
    while(count < n){
        safeFound=0;

        for(int i=0; i<n; i++){
            if(finish[i]==0){
                int safe = 1;
                for(j=0; j<r; j++){
                    if(need[i][j]>avail[j]){
                        safe = 0;
                        break;
                    }
                }

                if(safe){
                    for(k=0; k<r; k++){
                        avail[k] = avail[k] + alloc[i][k];
                    }
                    safe_sequence[count] = i;
                    finish[i] = 1;
                    safeFound = 1;
                    count++;
                }
            }
        }

        if(!safeFound){
            printf("System is in unsafe state! May occur the deadlock \n");
            return 0;
        }
    }
}

```

```
//print safe sequence
printf("Safe sequence is: ");
for(i=0; i<n; i++){
    printf("P%d ",safe_sequence[i]);
}

return 0;
```

```
user123@localhost:~/bankers
[user123@localhost ~]$ ls
abc.sh  awk  Desktop  Downloads  ifelse.sh  memory  page  Pictures  Public  REC  sample.sh  scheduling  SFA  system_calls  Videos
ab.sh  bankers  Documents  Folder  loops.sh  Music  pc  posneg.sh  qns.txt  sample.c  sample.txt  script  sh  Templates  while.sh
[user123@localhost ~]$ cd bankers
[user123@localhost bankers]$ vi bankers.c
[user123@localhost bankers]$ gcc bankers.c
[user123@localhost bankers]$ ./a.out
Enter number of processes: 4
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 0
Enter Max Matrix:
7 5 3
3 2 2
7 0 2
2 2 2
Enter Available Resources:
3 3 2
Safe sequence is: P1 P2 P3 P0 [user123@localhost bankers]$
```